

Not Your Average Agent: Play-style Conditioned Decision Transformers

Abstract—Automated playtesting agents can help game designers evaluate balance, difficulty, and player experience, but many existing reinforcement learning approaches collapse diverse player behaviors into a single policy. This limitation makes it difficult to systematically probe how different play styles interact with game mechanics and level design. In this work, we introduce StyleDT, a style-conditioned Decision Transformer designed to generate controllable, diverse gameplay behaviors. StyleDT models long-horizon player strategies as a continuous latent style representation learned from full gameplay trajectories. A conditional prior maps designer-specified behavioral controls to distributions over latent styles, which are injected as prompt tokens into a Decision Transformer policy. This architecture enables inference-time control over agent behavior without requiring reference trajectories at inference time.

We evaluate StyleDT in a MiniGrid-based playtesting environment featuring multiple viable strategies, including combat, stealth, and avoidance behaviors. Experimental results show that StyleDT can accurately reproduce different strategic behaviors, achieve strong style adherence, and match or exceed the performance of specialized single-style policies while operating as a unified model. Furthermore, the learned latent style representation enables reliable behavioral steering and generalizes to interpolated control configurations not present during training. These results demonstrate that trajectory-level style encoding combined with conditional latent priors provides an effective framework for controllable playtesting agents capable of exploring a broader distribution of player behaviors.

Index Terms—Reinforcement Learning, Play-style, Human-like Playtesting, Controllable Agents, Decision Transformer

I. INTRODUCTION

Reinforcement Learning (RL) has become an increasingly attractive tool to build gameplay agents in modern game development, driven by advances in offline learning, large-scale behavioral datasets, and sequence modeling architectures [1]. While recent research has demonstrated that RL-based agents can achieve strong performance in a wide range of games [2]–[4], their adoption in real-world production pipelines, particularly for game design and playtesting, remains limited by two persistent obstacles: (i) inability to capture diverse player styles and (ii) lack of behavioral controllability.

In practice, game level designers rely heavily on playtesting agents to evaluate balance, difficulty, and player experience under frequent content changes [5], [6]. Most ML-based gameplay agents are trained to optimize a single objective or to mimic an average behavior across the dataset, resulting in a generic agent that represents no particular player group. This one-size-fits-all paradigm fails to capture the rich diversity of player behaviors observed in real games, such as risk-taking versus cautious play, aggressive versus stealthy strategies, or

resource-driven versus avoidance-based decision-making. As a result, designers lack the ability to systematically probe how different play-styles interact with level layouts, item placements, and item distributions.

A natural solution is to build *style-aware* gameplay agents that can be steered to exhibit distinct behaviors at inference time. Prior work has explored style discovery and diversity in offline RL [7], [8]. However, these approaches typically yield a fixed set of discrete policies or skills, requiring either the explicit selection of a policy index or the use of reference trajectories at deployment. In playtesting settings, these requirements are often impractical, since designers may not have access to example trajectories and instead prefer to specify high-level behavioral preferences directly.

Recent advances in offline RL increasingly adopt transformer-based sequence models, framing RL as a conditional sequence modeling problem. Decision Transformer (DT) [9] learns policies from offline data by modeling trajectories autoregressively, conditioning actions on past states and desired returns. Extensions such as Prompting Decision Transformer [10] and Skill Decision Transformer [8] improve adaptability by conditioning on task-specific trajectory prompts or discovered skills. While these methods enhance generalization and enable task-level control, they do not address the problem of *controllable, style-aware playtesting* driven by designer-specified preferences, nor do they provide a mechanism to generate such styles without reference trajectories at inference time.

In this work, we propose STYLEDT, a controllable playtesting agent that enables designers to specify and sample diverse play-styles using a set of intuitive control configurations. STYLEDT combines three key components: (i) a full-trajectory encoder that learns a compact latent representation of long-horizon player strategies from offline gameplay data, (ii) a DT decoder conditioned on this latent as soft prompt tokens, and (iii) a learned conditional prior that maps designer-provided style controls such as risk tolerance and resource-seeking behavior to distributions over latent styles. This design enables coherent, style-consistent behavior at inference time *without requiring demonstration trajectories*, while still preserving diversity through latent sampling.

We evaluate STYLEDT in a MiniGrid-based playtesting environment featuring multiple viable strategies to reach a goal, including combat, stealth, and avoidance routes. Our experiments demonstrate that STYLEDT enables fine-grained control over agent behavior, produces diverse yet consistent play-styles under identical control settings, and generalizes

to interpolated style configurations not explicitly seen during training. By comparing against DT-based baselines, we show that both the full-trajectory style encoder and the conditional prior are essential for achieving controllable, realistic playtesting behavior ¹.

Contributions: Our main contributions are as follows:

- **STYLEDT:** We introduce a style-conditioned Decision Transformer architecture that models long-horizon play-styles via a latent representation learned from full trajectories, enabling controllable and diverse gameplay behavior within a single unified model.
- **Conditional Prior:** We present a conditional prior over style latents that enables inference-time control using designer-specified behavioral controls without reference trajectories or discrete policy selection.
- **Empirical Evaluation:** Through experiments in a custom MiniGrid environment, we demonstrate style controllability and generalization to interpolated control configurations.

II. RELATED WORK

We review prior work most relevant to STYLEDT and highlight how our approach differs from these lines of research, particularly in terms of inference-time controllability and applicability to playtesting.

A. Play-style Identification and Imitation

Identifying play styles from player data is a longstanding problem in game analytics and AI-assisted game design. Early work in this area focused on clustering trajectories or player features to discover recurring behavioral patterns, which can inform game balancing and content iteration. Such analyses are often performed without access to explicit game variables, relying instead on temporal similarity measures or trajectory alignment methods, such as Dynamic Time Warping, to group player behaviors into distinct styles [11], [12].

Beyond analysis, several methods leverage discovered styles to train gameplay agents that imitate specific player groups. For example, play-style-centric policy generation methods train multiple behavioral cloning agents, each specializing in a particular style cluster [7]. While effective at reproducing representative behaviors, these approaches typically result in a fixed set of discrete agents and require explicit policy selection at inference time.

On the topic of human-like play-style generation, de Woeillemont et al. [13] propose CARMI, a configurable RL agent conditioned on relative summary metrics that define play-styles with respect to the distribution of human players. Their approach trains a single agent to generate a continuum of play-styles by providing target play-mode values as inputs and shaping the reward to match these objectives, enabling generalization to previously unseen levels while relying only

on summarized human data rather than full trajectories. While this work demonstrates the effectiveness of metric-driven conditioning for automated testing, it relies on reward-based optimization and careful specification of target metrics.

Stylized Offline Reinforcement Learning (SORL) extends this paradigm by combining trajectory clustering with off-line policy optimization. SORL alternates between assigning trajectories to latent style indices and learning one policy per style using advantage-weighted updates, yielding multiple high-quality policies from heterogeneous datasets. This approach is well-suited for discovering and extracting diverse behaviors when styles are unknown a priori, but, like other multi-policy approaches, it does not support inference-time interpolation or control [14].

Recent work has also explored learning controllable and diverse player behaviours in multi-agent environments [15]. These approaches demonstrate that explicit mechanisms for behavioural control and diversity can be learned jointly within a single model. However, they typically rely on reward shaping or environment-specific training procedures, and do not provide a mechanism for inference-time control.

While prior style-centric methods focus on discovering or training separate policies for each style, STYLEDT addresses a complementary problem: enabling *controllable generation* of play styles at inference time. Instead of producing a discrete set of policies or requiring reference trajectories, our method learns a continuous latent style space and a conditional prior that maps designer-specified behavioral preferences to distributions over style latents. This allows a single agent to generate diverse, coherent behaviors without explicit policy selection.

B. Conditioned Offline RL and Transformer Policies

Recent advances in offline RL increasingly adopt transformer-based architectures, framing decision-making as a conditional sequence modeling problem. DT models trajectories autoregressively and conditions action prediction on past states and desired returns, enabling effective policy learning from offline data [9]. Several extensions of DT introduce conditioning mechanisms to improve adaptability and generalization. For example, PDT [10] prepends prompt embeddings to guide behavior in multi-task and few-shot settings, while Skill-DT [8] discovers discrete latent skills and conditions action generation on these skill embeddings and predicted skill transitions. These methods demonstrate that transformer decoders can effectively leverage both hard and soft conditioning signals to modulate behavior.

STYLEDT builds on these conditioning mechanisms by introducing a probabilistic latent style representation inferred from full trajectories and injected into a DT as soft prompt tokens. Unlike prior conditioned DT variants, which typically rely on discrete skills or explicit prompts provided at inference time, STYLEDT learns a conditional prior over style latents that enables inference-time control using continuous, interpretable designer inputs. This design allows STYLEDT to

¹The link to the code repository to reproduce all experiments, including all hyperparameters: https://anonymous.4open.science/t/player_model_dt-41E2/README.md

generate diverse play styles consistent with high-level preferences while maintaining long-horizon behavioral coherence.

In contrast to prior work on style discovery, discrete skill learning, or multi-policy generation, STYLEDT focuses on inference-time controllability for playtesting. By combining full-trajectory style encoding, a soft-prompted DT decoder, and a conditional prior driven by designer controls, our method enables controllable, diverse, and coherent gameplay behavior within a single unified model.

III. PRELIMINARIES

This section briefly reviews some basic concepts from Decision Transformer (DT) and Prompting Decision Transformer (PromptDT).

A. Decision Transformer (DT)

A DT [9] is a sequence model-based approach to RL that treats trajectory optimization as a supervised sequence prediction task. Instead of estimating value functions or learning an explicit policy, DT conditions action selection on desired return-to-go. In a Markov Decision Process (MDP), a trajectory is defined as a sequence of tuples $\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T)$, where $s_t \in \mathcal{S}$ is the state at timestep t , $a_t \in \mathcal{A}$ is the action taken, $r_t \in \mathbb{R}$ is the reward received, and T is the trajectory length. Instead of modeling a standard policy $\pi(a_t|s_t)$, DT conditions action selection on past states, actions, and future return-to-go $R_t = \sum_{t'=t}^T r_{t'}$. DT models the probability of the next action using a causal transformer $\pi(a_t|s_{\leq t}, a_{\leq t}, R_{\leq t})$, where inputs are tokenized as sequences $(R_1, s_1, a_1, R_2, s_2, a_2, \dots)$. This allows the model to generate actions autoregressively by conditioning on past trajectory data and future expected return, enabling goal-directed behavior in an offline RL setting.

B. Prompting Decision Transformer (PromptDT)

PromptDT [10] extends DT to a multi-task setting by incorporating task-specific demonstrations as additional inputs that guide the agent's behavior. These contextual inputs are commonly referred to as *prompts*, which serve as high-level behavioral guides, enabling PromptDT to control generalization to new tasks, improving flexibility compared to standard DT. PromptDT modifies the transformer input sequence as $(z, R_1, s_1, a_1, R_2, s_2, a_2, \dots)$, where z is composed of task-specific prompts of form $(R_1^*, s_1^*, a_1^*, R_2^*, s_2^*, a_2^*, \dots)$. PromptDT randomly samples trajectory segments from a set of expert demonstration trajectories $\mathcal{P}_i$ that constitute the prompt z for each training task $\mathcal{T}_i \in \mathcal{T}^{\text{train}}$.

IV. METHOD

We present STYLEDT, a controllable playtesting agent that generates style-consistent gameplay behavior conditioned on designer-specified controls. STYLEDT combines (i) a full-trajectory style encoder, (ii) a conditional latent prior, and (iii) a DT decoder with soft prompt conditioning. An overview of the architecture is shown in Figure 1.

A. Problem Setting

We consider an offline dataset $\mathcal{D} = \{(\tau_i, c_i)\}_{i=1}^N$, where each trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ is generated by a particular play-style, and $c \in \mathbb{R}^d$ is a vector of domain-relevant behavioral controls that can provide a high-level description of different styles. Examples of such control variables include risk tolerance, stealth preference, and resource collection preference. The controls c are provided during training and are assumed to be available at inference time, reflecting the designer's intent.

We aim to learn policy $\pi(a_t | s_{\leq t}, a_{\leq t}, R_{\leq t}, z)$ that: (i) produces coherent long-horizon behavior consistent with specific styles represented by the latent variable z , (ii) allows inference-time control via c without requiring reference trajectories as input to the model, and (iii) supports diversity by sampling multiple behaviors consistent with the same controls. To achieve this, STYLEDT factorizes behavior generation into three components:

- 1) *Style encoder* $q_\phi(z | \tau)$ that maps a full trajectory to a latent style representation.
- 2) *Conditional prior* $p_\psi(z | c)$ that maps specified controls to a distribution over the style latents.
- 3) *DT decoder* $p_\theta(a_t | s_{\leq t}, a_{\leq t}, R_{\leq t}, z)$ that generates actions conditioned on the style latent.

In the rest of this section, we explain these components.

B. Style Encoder

The style encoder compresses an entire trajectory of states and actions into a latent variable that captures long-horizon strategic behaviors, referred to as *styles* (part B in Figure 1). Given a full trajectory τ , we first embed states and actions using learned embedding layers and arrange them as a sequence of tokens augmented with positional (timestep) embeddings. This sequence is then processed by a Transformer encoder, producing token representations that capture temporal dependencies across the trajectory.

Since trajectories may vary in length, we apply mean pooling to obtain a fixed-length representation. This representation is passed through linear projection layers that output the parameters of a latent distribution. In our implementation, we use a Gaussian posterior:

$$q_\phi(z | \tau) = \mathcal{N}(\mu_\phi(\tau), \text{diag}(\sigma_\phi^2(\tau)))$$

where q_ϕ denotes the encoder model, μ_ϕ and σ_ϕ^2 are the mean and variance of the Gaussian $\mathcal{N}$. The latent variable z is sampled using the “reparameterization trick”, $z = \mu_\phi(\tau) + \sigma_\phi(\tau) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$. Encoding full trajectories ensures that z captures global strategy choices such as route commitment or risk management that are not recoverable from a short trajectory context window.

C. Conditional Prior over Styles

To enable inference-time control without requiring demonstration trajectories as input, we learn a conditional prior that maps designer-specified behavioral controls to a distribution

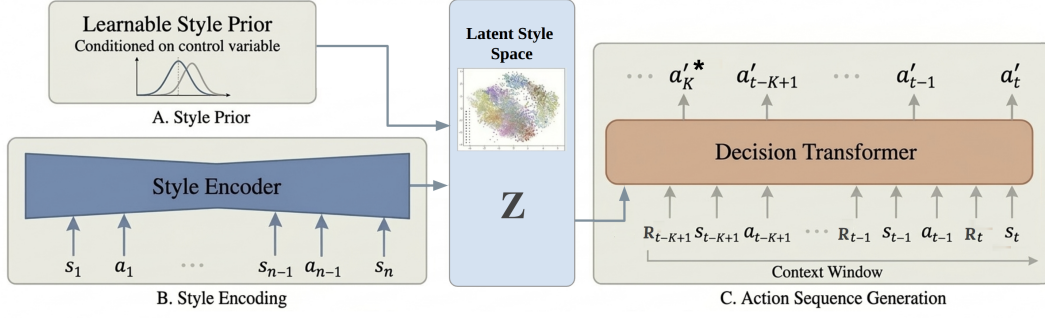


Fig. 1. Overview of STYLEDT. A trajectory encoder maps full gameplay trajectories to a latent style variable z . A conditional prior learns to map control variables to a distribution over styles. The sampled style embedding is injected as a prompt into a DT, enabling the generation of style-consistent behavior.

over latent styles. Let $c \in \mathbb{R}^d$ denote a vector of high-level control variables describing the desired play-style (e.g., risk tolerance, stealth preference, or resource-seeking behavior). The conditional prior is defined as:

$$p_\psi(z | c) = \mathcal{N}(\mu_\psi(c), \text{diag}(\sigma_\psi^2(c))),$$

where $\mu_\psi(c)$ and $\sigma_\psi^2(c)$ are produced by a lightweight multi-layer perceptron (MLP) parameterized by ψ . The MLP takes the control vector c as input and outputs the mean and log-variance parameters of the latent Gaussian distribution.

During training, the conditional prior is aligned with the encoder posterior $q_\phi(z | \tau)$ by minimizing the Kullback–Leibler (KL) divergence:

$$D_{\text{KL}}(q_\phi(z | \tau) \| p_\psi(z | c)),$$

encouraging the prior to produce latent style representations consistent with those inferred from trajectories. This alignment allows the model to learn a mapping between interpretable control variables and regions of the latent style space.

At inference time, designers specify a control vector c to the learned prior $p_\psi(z | c)$ from which a style latent z is sampled. This sampled latent variable is then used to condition the policy, thereby enabling direct control over the agent’s behaviour without requiring reference trajectories.

D. DT Decoder with Style Prompts

We adopt the DT as the policy decoder. Following the DT formulation, trajectories are modeled as sequences of tokens $(R_1, s_1, a_1, R_2, s_2, a_2, \dots)$, where R_t denotes the return-to-go at timestep t . The model predicts actions by conditioning on past states, actions, and desired returns. To incorporate play-style information, we condition the Transformer on the latent style variable z . Specifically, the latent style embedding is projected into the token embedding space and prepended to the input sequence as a set of *style prompt tokens*. These tokens act as global conditioning signals that influence the action generation. The resulting decoder defines a style-conditioned policy as follows:

$$p_\theta(a_t | s_{\leq t}, a_{< t}, R_{\leq t}, z).$$

Injecting the style embedding as prompts to the model provides the Transformer with style information without modifying the core DT architecture. This mechanism encourages the policy to generate behavior that remains consistent with the specified play-style over long horizons.

E. Training Objective

STYLEDT is trained using an offline objective that combines behavior cloning and latent regularization. For a trajectory τ with controls c , we minimize:

$$\begin{aligned} \mathcal{L} = \mathbb{E}_{z \sim q_\phi(z | \tau)} \left[\sum_t \mathcal{L}_{\text{BC}}(a_t, \hat{a}_t) \right] \\ + \beta \text{KL}(q_\phi(z | \tau) \| p_\psi(z | c)), \end{aligned}$$

where $\mathcal{L}_{\text{BC}}$ is a supervised imitation loss (cross-entropy for discrete actions or mean squared error for continuous actions), and β controls the strength of the KL regularization.

We train the decoder using fixed-length context windows sampled from trajectories, while the encoder always observes the full episode. This mirrors inference-time usage, where the policy operates with limited history but is conditioned on a global style latent.

F. Inference and Playtesting

At inference time, designers specify a control vector c that reflects the desired behavioral preferences. A style latent is sampled from the learned prior $p_\psi(z | c)$, and used as style prompt tokens to condition the DT during rollout. Sampling multiple latents for the same c yields diverse yet style-consistent playthroughs, enabling robust playtesting under controlled behavioral variations. This sampling mechanism also enables exploration of interpolated control configurations; for example, blending high-risk and high-resource-seeking preferences to generate more stochastic yet style-consistent behaviors.

V. EXPERIMENTS

In this section, we evaluate STYLEDT through a series of controlled experiments designed to assess task performance,

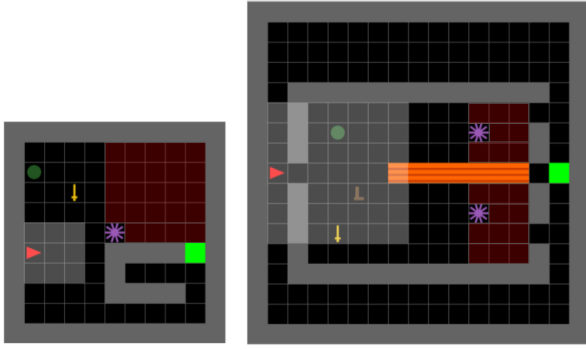


Fig. 2. **MiniGrid environment simulating a simple stealth game.** The red triangle represents the player, who has a fixed field of view. The purple stars denote the enemy. The yellow sword is a collectible weapon that allows the player to eliminate the enemy, the green circle represents a camouflage item that helps the player avoid detection, and the brown boots help the agent to traverse through the lava tunnel. The goal is marked by a green square. The highlighted red area indicates the region that can be detected by the enemy.

control fidelity, and generalization. We first describe the experimental setup, including the environment, offline dataset construction, and implementation details. We then introduce the baseline methods used for comparison, including both DT-based variants and style-specific behavioral cloning models. We also present the metric used to compare the methods. Finally, we present quantitative and qualitative results analyzing control adherence and style consistency, task performance measured by success rate, and generalization under interpolated and out-of-distribution control configurations.

A. Environments and Datasets

We evaluate STYLEDT in custom MiniGrid-based environments [16] (Figure 2) designed to capture multiple viable play strategies. Two environments are presented: a simpler, smaller map and a larger, more complex map. The environments consist of a grid world containing an agent, a goal location, an enemy, and some optional items: a weapon, a camouflage, and an iron boot. At each environment reset, the collectible items are randomly placed on a different tile within a predefined region of the map. Additionally, in the smaller map, the agent and goal locations are perturbed slightly between resets to introduce further variability in the environment. The agent’s objective is to reach the goal while avoiding death. The enemy has a fixed position and a region-of-sight, and the agent is terminated if it enters that region while the enemy is alive. The weapon can be collected to eliminate the enemy in combat, enabling safe traversal through its region-of-sight, while the camouflage allows the agent to remain undetected even when entering those visible tiles. In the larger map, another possibility is to take the iron boots and go through the risky lava tunnel in the middle of the map. Moreover, the environment includes alternative routes that allow the agent to bypass the risky scenarios at the cost of longer paths to the goal. The episode is terminated upon: entering the lava without boot, entering the enemy detection zone without elimination, or upon timeout (130 steps).

The offline dataset consists of gameplay trajectories generated using pre-trained online RL models under distinct strategic behaviors: (i) *combat* strategy, where the agent first collects the sword item and uses it to eliminate the enemy, and then proceeds through the enemy’s region of sight to reach the goal, (ii) *stealth* strategy, where the agent collects the camouflage item, allowing it to traverse the enemy’s detection region without being detected, (iii) *daredevil* strategy (only present in the large map) where the agent collects a boot to enable safe traversal through the lava tunnel, and (iv) *bypass* strategy, where the agent avoids interacting with all items and instead follows an alternative route that circumvents the enemy’s detection region entirely, typically resulting in a longer but safer path to the goal. Each trajectory records the sequence of states, actions, and termination signals, and is labeled with a high-level behavioral control vector provided by the environment.

B. Baselines

We compare STYLEDT with four baseline approaches, each representing alternative strategies for incorporating style information into offline policy learning.

Stylized Offline Reinforcement Learning (SORL). We include Stylized Offline Reinforcement Learning (SORL) [14] as an additional baseline. SORL learns multiple policies corresponding to different behavioural styles by clustering trajectories and optimizing one policy per cluster. At inference time, a specific style is selected by choosing the corresponding policy. This baseline evaluates whether explicitly learning separate style-specific policies can match the controllability and performance of a unified style-conditioned model.

Prompting Decision Transformer (PromptDT). As a first baseline, we implement PromptDT [10] adapted to the play-style setting. PromptDT conditions action generation on short trajectory segments prepended as prompts to the input sequence. In our setting, the prompt consists of a short section of an expert trajectory corresponding to a specific strategy (e.g., weapon-based, camouflage-based, or bypass). The model, therefore, infers the intended style implicitly from demonstration tokens rather than from an explicit control variable. During training, prompt segments are randomly sampled from trajectories belonging to the same style as the target rollout window. At inference time, a representative trajectory segment for the desired style is provided as a prompt. This baseline evaluates whether direct trajectory-based prompting is sufficient to recover coherent style-conditioned behavior.

Control-Prompt Decision Transformer (ControlDT). Our second baseline replaces trajectory prompts with the high-level control vector c . Instead of prepending demonstration tokens, we prepend an embedding of the control vector to the DT input sequence as a soft prompt. ControlDT provides the model with explicit behavioral descriptors, but does not learn

a latent style representation or a conditional prior. In contrast to STYLEDT, ControlDT must directly map control variables to low-level actions without an intermediate trajectory-level encoding. This baseline isolates the contribution of the learned latent style space and conditional prior.

Behavioral Cloning (BC) per Style. As a non-Transformer baseline, we train independent BC policies, one for each style. Each model is trained using supervised imitation learning on trajectories corresponding to a single strategy. At inference time, the appropriate policy is selected based on the desired style. While this approach avoids the need for conditioning mechanisms, it requires training and maintaining multiple separate policies and does not support interpolation between styles. This baseline evaluates whether STYLEDT can match or exceed specialized single-style agents while maintaining controllability in a unified model.

C. Evaluation Metrics

We evaluate the methods across three complementary dimensions: *online rollout performance*, *behavioral control fidelity*, and *latent space quality*. Not all dimensions are applicable to every baseline; we note where a metric is method-specific.

Rollout Metrics. For each method and target style $s \in \{\text{combat}, \text{stealth}, \text{daredevil}, \text{bypass}\}$, we execute rollouts in the environment with randomized layout and report the following per-style statistics, aggregated over episodes:

- *Success Rate (SR)*: the fraction of episodes in which the agent reaches the goal before the time limit.
- *Style Achievement Rate (SAR)*: among successful episodes, the fraction in which the agent’s realized behavioral style matches the intended target style s , as determined by the environment’s episode summary.
- *Detection Rate (DR)*: the fraction of episodes terminated by enemy detection, measuring stealth compliance.
- *Average Enemy Distance (AED)*: the mean Euclidean distance to the enemy over timesteps in successful episodes, serving as a proxy for safety-seeking behavior.
- *Average Path Efficiency (APE)*: the ratio of the shortest-path length to the actual path length in successful episodes, measuring directness of navigation.
- *Weapon/Camouflage Usage Rate (WUR, CUR)*: the fraction of successful episodes in which the weapon or camouflage item was collected, measuring resource-seeking behavior.

Control Fidelity. We define $\mathbf{c}$ as a behavioral control vector with three dimensions: (i) *risk_tolerance* reflects the agent’s willingness to approach enemies (computed from the normalized minimum distance to the enemy), (ii) *resource_preference* measures the tendency to collect items such as the weapon or camouflage (based on the fraction of items collected), and (iii) *commitment* captures how directly the agent follows its chosen path toward the goal (measured

by the ratio of forward movement steps to total steps). Control fidelity measures how faithfully each control dimension drives its intended behavioral outcome. Specifically, we report the mean absolute Spearman correlation $\bar{\rho} = \frac{1}{K} \sum_k |\rho_k|$ as summary of control fidelity, where $\rho_k = 1 - \frac{6 \sum_{i=1}^N d_i^2}{N(N^2-1)}$, and $d_i = \text{rank}(c_k^{(i)}) - \text{rank}(b_k^{(i)})$ denotes the difference between the ranks of the control values $c_k^{(i)}$ and the corresponding behavioral measurements $b_k^{(i)}$ over N samples.

Generalization to Interpolated Controls. To evaluate generalization, we test the model on interpolated control vectors (e.g., blending weapon and camouflage preferences) that often lie in regions of the control space not observed during training. We study generalization by analyzing shifts in the distribution of generated behaviors when the model is conditioned on unseen control configurations.

VI. RESULTS & ANALYSIS

A. Model Performance

Tables I, II, and III report rollout metrics for the three target strategies (bypass, weapon, and camouflage) across all evaluated models. Overall, STYLEDT achieves performance comparable to or exceeding specialized baselines while maintaining strong style adherence, demonstrating that a single style-conditioned model can reproduce diverse behaviors without requiring separate policies.

TABLE I
PER-STYLE ROLLOUT METRICS — **BYPASS**.

Metric	StyleDT	BC	PromptDT	ControlDT	SORL
SR	1.00±0.00	1.00±0.00	0.99±0.01	1.00±0.00	1.00±0.00
SAR	0.96±0.08	1.00±0.00	0.94±0.02	0.93±0.08	0.96±0.08
DR	0.00±0.00	0.00±0.00	0.01±0.02	0.00±0.00	0.00±0.00
AED	6.23±0.05	6.24±0.04	5.91±0.13	6.24±0.02	6.28±0.07
APE	0.79±0.00	0.79±0.02	0.78±0.00	0.79±0.01	0.78±0.02
WUR	0.01±0.02	0.00±0.00	0.06±0.02	0.00±0.00	0.00±0.00
CUR	0.04±0.08	0.00±0.00	0.00±0.00	0.06±0.08	0.04±0.08

TABLE II
PER-STYLE ROLLOUT METRICS — **WEAPON**.

Metric	StyleDT	BC	PromptDT	ControlDT	SORL
SR	0.96±0.00	0.96±0.08	0.93±0.07	0.93±0.04	1.00±0.00
SAR	0.97±0.02	1.00±0.00	0.90±0.07	0.93±0.03	0.00±0.00
DR	0.02±0.02	0.00±0.00	0.06±0.05	0.01±0.01	0.00±0.00
AED	0.19±0.10	0.00±0.00	0.58±0.43	0.40±0.19	6.32±0.06
APE	0.55±0.01	0.55±0.01	0.58±0.02	0.57±0.01	0.78±0.02
WUR	0.98±0.02	1.00±0.00	0.90±0.07	0.94±0.04	0.00±0.00
CUR	0.02±0.02	0.00±0.00	0.09±0.07	0.05±0.05	0.00±0.00

Bypass Strategy. In the bypass setting (Table I), all models achieve near-perfect success rates, indicating that the task itself is relatively easy once the correct route is chosen. However, differences emerge in *style adherence*. STYLEDT achieves a high style achievement rate (SAR=0.96), closely approaching the oracle behavioral cloning baseline (SAR=1.0), while outperforming PromptDT (0.94) and ControlDT (0.93). PromptDT exhibits occasional style leakage, as evidenced by a higher weapon usage rate (0.06) despite the bypass strategy

TABLE III
PER-STYLE ROLLOUT METRICS — CAMOUFLAGE.

Metric	StyleDT	BC	PromptDT	ControlDT	SORL
SR	1.00±0.00	0.92±0.10	0.99±0.01	0.99±0.01	0.96±0.08
SAR	1.00±0.00	1.00±0.00	0.99±0.01	0.78±0.05	0.00±0.00
DR	0.00±0.00	0.00±0.00	0.01±0.01	0.01±0.01	0.00±0.00
AED	6.00±0.01	5.99±0.03	5.93±0.11	6.06±0.03	0.00±0.00
APE	0.76±0.01	0.77±0.02	0.76±0.01	0.78±0.01	0.57±0.02
WUR	0.00±0.00	0.00±0.00	0.01±0.01	0.78±0.01	1.00±0.00
CUR	1.00±0.00	1.00±0.00	0.99±0.01	0.78±0.05	0.00±0.00

ideally avoiding item interactions. In contrast, STYLEDT maintains near-zero weapon usage while preserving safe navigation behavior, as reflected in its high average enemy distance and zero detection rate. These results indicate that STYLEDT captures the long-horizon avoidance behavior characteristic of bypass strategies, rather than simply reaching the goal by any means.

Weapon Strategy. The weapon-focused strategy (Table II) requires the agent to collect the sword and eliminate the enemy before proceeding. In this setting, STYLEDT achieves strong performance with high style fidelity and resource usage (WUR=0.98), approaching the oracle BC baseline (1.0) and outperforming both PromptDT and ControlDT. While BC achieves slightly cleaner behavior due to its single-style specialisation, STYLEDT maintains comparable success rates and style adherence while operating as a unified multi-style model. PromptDT shows weaker style consistency, reflected in lower SAR and higher camouflage usage, suggesting that short prompt segments may not always provide sufficient context to enforce a specific strategic pattern. ControlDT performs moderately well but shows less consistent behavior, with slightly lower style adherence and higher variability in enemy distance. This indicates that directly conditioning the policy on control variables without a structured latent representation makes it harder for the model to learn coherent long-horizon strategies.

Camouflage Strategy. The camouflage setting (Table III) highlights the advantages of STYLEDT. The model achieves perfect success and style adherence (SR=1.0, SAR=1.0) while consistently using camouflage items (CUR=1.0). This performance matches the Oracle BC baseline while surpassing both PromptDT and ControlDT. Notably, ControlDT struggles significantly with this style, exhibiting a lower SAR (0.783) and incorrectly collecting weapons in a large fraction of episodes. This behavior suggests that the model fails to consistently associate control inputs with the intended strategic behavior. In contrast, STYLEDT maintains coherent stealth-oriented navigation, as reflected in its high enemy distance and zero detection rate.

Across all three strategies, the results demonstrate three key findings. First, STYLEDT matches or approaches the performance of specialized single-style policies while maintaining the flexibility of a unified model. Second, PromptDT provides

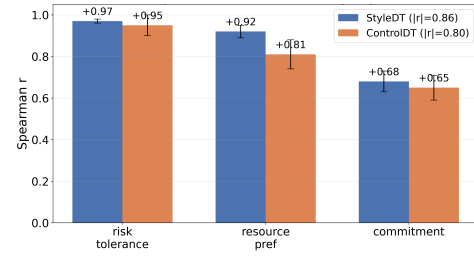


Fig. 3. Control Fidelity based on Spearman correlation.

some behavioral guidance but is insufficient for enforcing consistent long-horizon strategies. Third, SORL achieves strong task performance but fails to distinguish between weapon and camouflage strategies, as reflected in mismatched resource usage (WUR/CUR) and near-zero style achievement rates. This is due to the high overlap between these behaviours, which differ only in a small number of actions, making them difficult to separate based on trajectory likelihood. This highlights a limitation of multi-policy approaches, which rely on separability in the action distribution. Finally, ControlDT struggles to reliably translate discrete high-level behavioral controls into coherent policies, highlighting the importance of the learned latent style representation. Overall, these results indicate that the combination of a trajectory encoder and a conditional latent prior enables STYLEDT to capture the underlying structure of player strategies. This allows the model to generate style-consistent behavior while avoiding the need for multiple specialized agents.

B. Control Fidelity

Figure 3 compares how faithfully STYLEDT and ControlDT translate control variables into behavioral outcomes. STYLEDT consistently achieves higher Spearman correlations across all evaluated dimensions, indicating stronger alignment between the prompted controls and the generated behavior. These results suggest that the latent style representation learned by STYLEDT enables a more reliable behavioral steering than directly mapping control variables to actions.

C. Generalization Ability

To evaluate whether the learned style representations generalize beyond the control distribution observed during the training, we designed an *interpolation* experiment in which agents are conditioned on control vectors that are linear blends of two canonical style prototypes rather than any single training vector. These blended vectors may not appear in the training dataset and therefore represent out-of-distribution inputs for the model. We constructed three interpolation sequences: bypass $\leftrightarrow$ camouflage, weapon $\leftrightarrow$ camouflage, and weapon $\leftrightarrow$ bypass, each containing four steps ranging from pure style A (100%/0%) through two intermediate mixtures (60%/40% and 40%/60%) to pure style B (0%/100%). For each control vector, we executed N rollouts across five random seeds. We report the distribution of achieved styles.

As shown in Figures 4, 5, and 6, STYLEDT successfully captures this behavioral shift in a meaningful way across

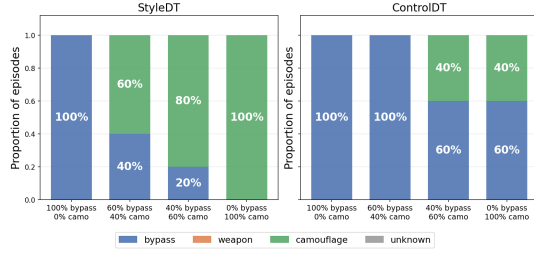


Fig. 4. Style-interpolation transitions: **bypass** ↔ **camouflage**

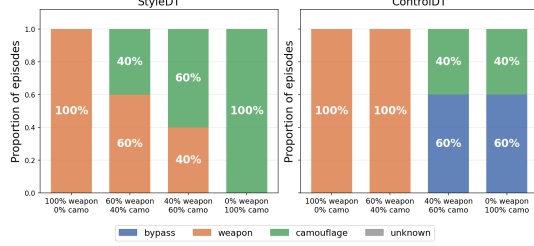


Fig. 5. Style-interpolation transitions: **weapon** ↔ **camouflage**

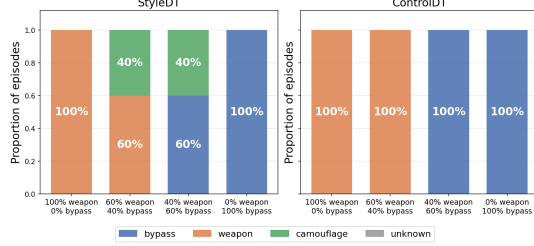


Fig. 6. Style-interpolation transitions: **weapon** ↔ **bypass**

most interpolation sequences; except for the weapon ↔ bypass sequence (Figure 6) which collapse to the camouflage style at intermediate blends due to the proximity of these control vectors in the dataset. ControlDT largely fails to respond to the interpolated inputs, collapsing to a dominant style regardless of the blend ratio. We attribute this disparity to the qualitative difference in how the two models represent style: ControlDT is trained to map a fixed set of discrete control vectors to corresponding behaviors and therefore has no mechanism to generalize to unseen regions of control space. STYLEDT, by contrast, learns a continuous latent space through its trajectory encoder, embedding styles as a structured manifold rather than a lookup table; interpolated control vectors therefore map naturally to coherent intermediate points on this manifold. These findings demonstrate that the encoder component is essential. It is the encoder’s role in structuring the latent space during training that enables the model to generalize.

VII. CONCLUSION

In this work, we introduce STYLEDT, a style-conditioned Decision Transformer (DT) that enables controllable, diverse gameplay behavior for automated playtesting. Unlike prior approaches that rely on discrete policies, demonstration prompts, or fixed behavior clusters, STYLEDT models play styles as a

continuous latent representation learned from full trajectories. By combining a trajectory encoder, a conditional prior over style latents, and a prompt-conditioned DT decoder, STYLEDT allows game designers to control agent behavior through interpretable high-level variables without requiring reference trajectories at inference time.

Experimental results demonstrate that STYLEDT can reliably reproduce multiple strategic behaviors, maintain strong style adherence, and match the performance of specialized single-style policies while operating as a unified model – outperforming multi-policy approaches such as SORL on style adherence while additionally supporting interpolation across styles. The results further show that the learned latent representation enables more reliable behavioral steering compared to directly conditioning on control variables. In addition, STYLEDT exhibits strong generalization to interpolated control configurations that were not present in the training data, suggesting that the learned latent style space captures meaningful structure in player strategies. Overall, these findings show that combining trajectory-level style encoding with conditional latent priors enables effective controllable behavior generation in offline RL, supporting scalable playtesting across diverse player behaviors.

A. Limitations and Future Work

The current evaluation is limited to a small grid-world environment with three discrete strategies. An important next step is to test STYLEDT in more complex environments. Evaluating the method on real player data would also strengthen its relevance for production playtesting.

ACKNOWLEDGMENT

Generative AI tools were used to assist with figure styling and prose editing in Sections I and II.

REFERENCES

- [1] X. Xu et al., “A survey on game playing agents and large models: Methods, applications, and challenges,” *arXiv preprint arXiv:2403.10249*, 2024.
- [2] C. Berner et al., “Dota 2 with large scale deep reinforcement learning,” *arXiv preprint arXiv:1912.06680*, 2019.
- [3] O. Vinyals et al., “Grandmaster level in starcraft ii using multi-agent reinforcement learning,” *nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [4] A. Badia et al., “Agent57: Outperforming the atari human benchmark,” in *ICML*. PMLR, 2020, pp. 507–517.
- [5] M. Mario et al., “A review of the trends, methods, and impacts of dynamic game balancing,” *IJCDs*, vol. 16, no. 1, pp. 1–13, 2024.
- [6] H. Jeon et al., “Raidenv: Exploring new challenges in automated content balancing for boss raid games,” *IEEE ToG*, vol. 16, no. 3, pp. 645–658, 2023.
- [7] B. Ingram et al., “Creating diverse play-style-centric agents through behavioural cloning,” in *AAAI AIIDE*, vol. 19, no. 1, 2023.
- [8] S. Sudhakaran et al., “Skill decision transformer,” *arXiv preprint arXiv:2301.13573*, 2023.
- [9] L. Chen et al., “Decision transformer: Reinforcement learning via sequence modeling,” *NeurIPS*, vol. 34, pp. 15 084–15 097, 2021.
- [10] M. Xu et al., “Prompting decision transformer for few-shot policy generalization,” in *ICML*. PMLR, 2022, pp. 24 631–24 645.
- [11] M. Ferguson et al., “Player style clustering without game variables,” in *ACM FDG*, 2020, pp. 1–4.
- [12] —, “Imitating playstyle with dynamic time warping imitation,” in *ACM FDG*, 2022, pp. 1–11.

- [13] P. de Woillemont et al., “Automated play-testing through rl based human-like play-styles generation,” in *AAAI AIIDE*, vol. 18, no. 1, 2022, pp. 146–154.
- [14] Y. Mao et al., “Stylized offline reinforcement learning: Extracting diverse high-quality behaviors from heterogeneous datasets,” in *ICLR*, 2024.
- [15] A. Cilan and A. Özgövde, “Learning controllable and diverse player behaviors in multi-agent environments,” *arXiv preprint arXiv:2512.10835*, 2025.
- [16] M. Chevalier-Boisvert et al., “Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks,” *NeurIPS*, vol. 36, pp. 73 383–73 394, 2023.